

3 РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ

3.1 Разработка объектов базы данных в выбранной СУБД

В качестве СУБД была выбрана PostgreSQL. Данная СУБД распространяется бесплатно, умеет работать с составными запросами, большими нагрузками и критическими данными. [4]

В результате, основываясь на спроектированной ER-диаграмме, было разработано 7 таблиц.

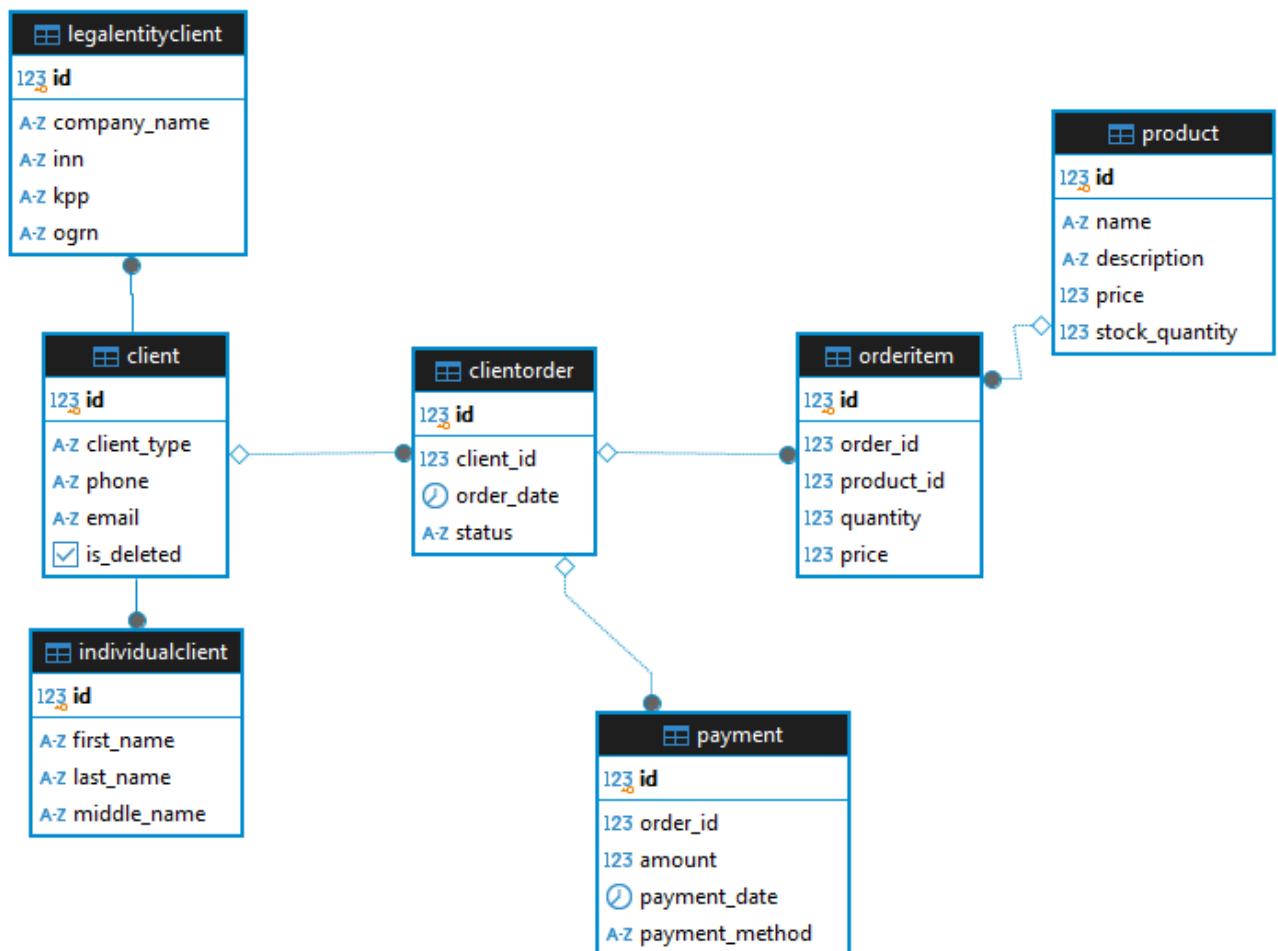


Рисунок 3.1 – Разработанные объекты

client – базовая таблица для клиентов.

- client_type указывает на тип клиента (individual или legal_entity).
- is_deleted используется для мягкого удаления.
- Связана один-к-одному с individualclient (физ. лица) и legalentityclient (юр. лица).

individualclient – данные о физических лицах.

- Хранит ФИО клиента.
- Использует id из client как PRIMARY KEY.
legalentityclient – данные о юридических лицах.
- Содержит название компании, ИНН, КПП, ОГРН.
- id также является PRIMARY KEY и FOREIGN KEY для client.
clientorder – заказы клиентов.
- Связан с client через client_id.
- Хранит дату заказа и статус.
- Связан с orderitem (товары в заказе) один-ко-многим.
orderitem – товары в заказе.
- Связан с clientorder через order_id и с product через product_id.
- Хранит количество товара и его цену в заказе.
product – каталог товаров.
- Описание, цена, количество на складе.
- Связан с orderitem один-ко-многим.
payment – информация об оплате заказа.
- Связан с clientorder через order_id.
- Хранит сумму платежа, дату и метод оплаты.

3.2 Реализация функциональной архитектуры программного средства

Для реализации функциональной архитектуры программного средства был использован язык программирования Python и такие библиотеки, как: PySide6, SQLAlchemy, Reportlab, Bcrypt. [5]

Совместно с предпринимателем было принято решение о том, что разработка макетов, работа над визуальной частью программного средства будет выполнена вне рамок курсовой работы.

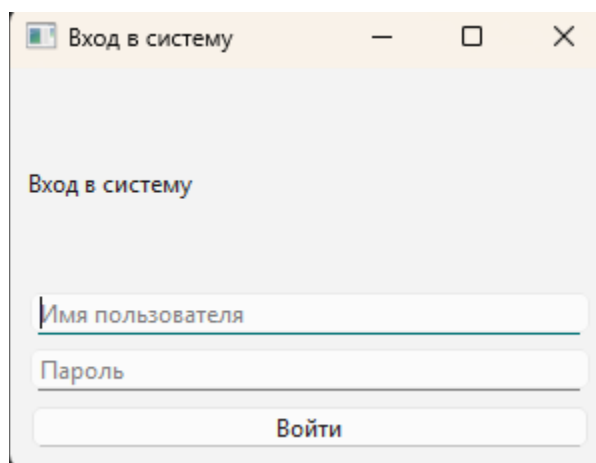


Рисунок 3.2 – Форма входа в систему

Форма LoginWindow, представленная на рисунке 3.2, является стартовой точкой приложения. Она позволяет пользователям авторизоваться под своими учётными записями. При успешной авторизации будет открыто окно, соответствующее роли пользователя. Авторизация представлена на листинге 3.1. Метод проверяет правильность логина и пароля, а также роль пользователя. В зависимости от роли метод открывает окно администратора или пользователя. [6] [7]

Листинг 3.1 – Авторизация пользователя

```
def login(self):
    username = self.username_input.text().strip()
    password = self.password_input.text().strip()

    if not username or not password:
        QMessageBox.warning(self, "Ошибка", "Введите имя
пользователя и пароль")
        return

    # Проверка пользователя в базе
    session = create_connection()
    user =
session.query(User).filter_by(username=username).first()

    if user and user.check_password(password):
        # Получаем роль пользователя
        user_role =
session.query(UserRole).filter_by(user_id=user.id).first()
        role =
session.query(Role).filter_by(id=user_role.role_id).first() if
user_role else None
        role_name = role.name if role else "Basic User"

    # Приводим имя роли к значению, используемому в коде
    role_mapping = {
        "Basic User": "basic",
```

```

        "Administrator": "admin",
        "Sales Manager": "sales_manager",
        "Worker": "worker",
        "Accountant": "accountant",
        "Director": "director"
    }
    role_value = role_mapping.get(role_name, "basic")

    # В зависимости от роли открываем соответствующий
интерфейс
    if role_value == "admin":
        self.admin_window = AdminWindow(session, user,
role_value)
        self.admin_window.show()
    else:
        self.user_window = UserWindow(session, user,
role_value)
        self.user_window.show()
    self.close()

```

Продолжение листинга 3.1

```

    else:
        QMessageBox.critical(self, "Ошибка", "Неверное имя
пользователя или пароль")
        session.close()

```

Форма AdminWindow, представленная на рисунке 3.3, предназначена для управления пользователями системы. Администратор может создавать, редактировать и удалять пользователей. Для создания и редактирования пользователей форма взаимодействует с диалоговыми окнами CreateUserDialog и EditUserDialog.

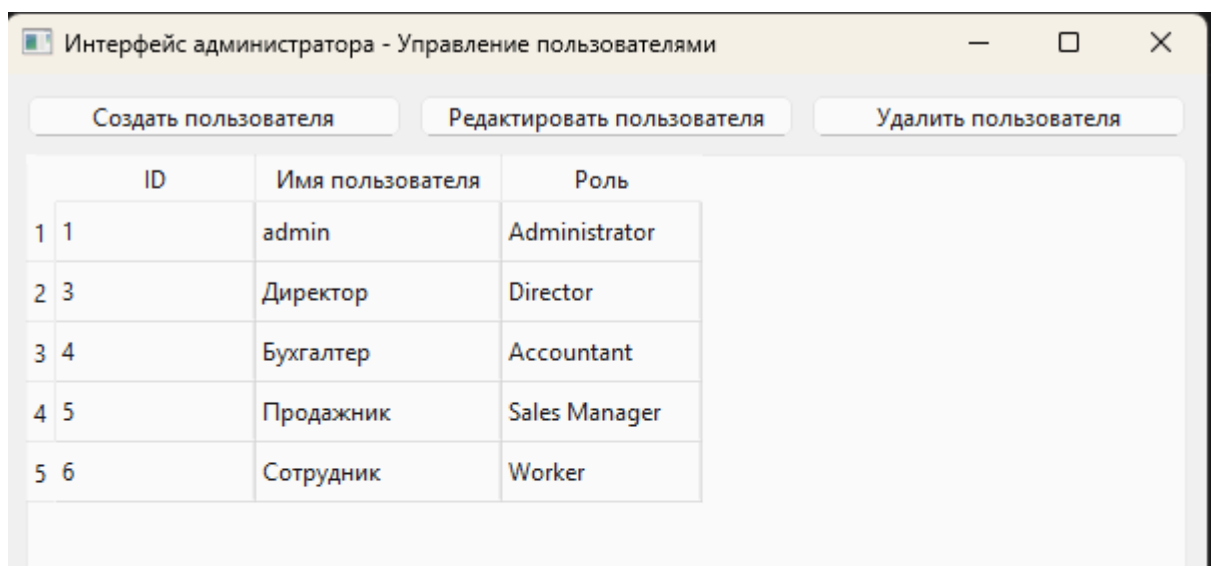


Рисунок 3.3 – Интерфейс администратора

На листинге 3.2 представлен фрагмент кода для создания пользователя. Метод проверяет уникальность имени пользователя, пароль шифруется при помощи метода из модели User. Данные пользователя сохраняются в таблице User, а его роль – в таблицу UserRole.

Листинг 3.2 – Создание пользователя

```
def create_user(self):
    try:
        username = self.username_input.text().strip()
        password = self.password_input.text().strip()
        role_name = self.role_combo.currentText()

        if not username or not password:
            QMessageBox.warning(self, "Ошибка", "Заполните все
поля")
            return

        if
self.session.query(User).filter_by(username=username).first():
            QMessageBox.warning(self, "Ошибка", "Пользователь с
таким именем уже существует")
            return

        role =
self.session.query(Role).filter_by(name=role_name).first()
        if not role:
            QMessageBox.warning(self, "Ошибка", f"Роль
{role_name} не найдена в базе данных")
            return

        new_user = User(
            username=username,
            password_hash=User.hash_password(password)
        )
        self.session.add(new_user)
        self.session.flush()

        user_role = UserRole(user_id=new_user.id,
role_id=role.id)
        self.session.add(user_role)

        self.session.commit()
        QMessageBox.information(self, "Успех", "Пользователь
успешно создан")
        self.accept()
    except Exception as e:
        self.session.rollback()
        QMessageBox.critical(self, "Ошибка", f"Ошибка при
создании пользователя: {e}")
```

Форма UserWindow, представленная на рисунке 3.4 предоставляет функционал для работы с клиентами и заказами. Интерфейс адаптируется в зависимости от роли пользователя.

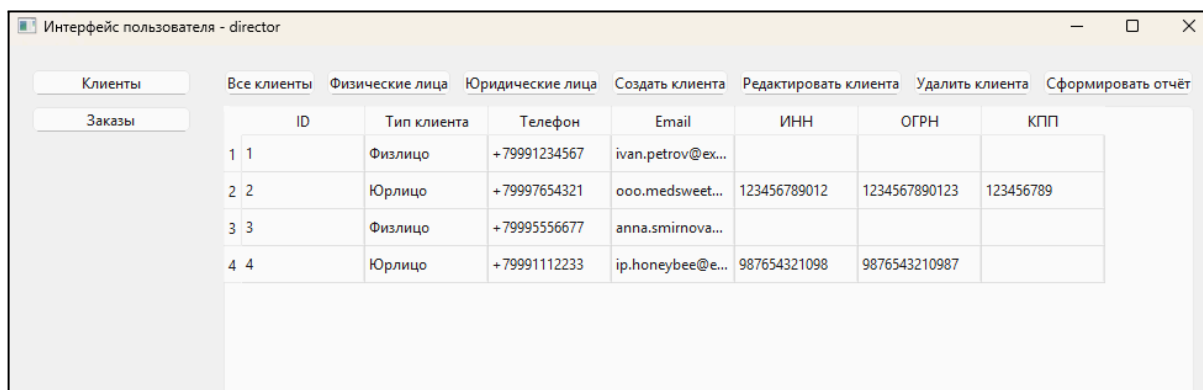


Рисунок 3.4 – Интерфейс пользователя

На листинге 3.3 представлен метод `create_order`, который создаёт новый заказ. Проверяется наличие товара и при успешном создании заказа остаток товара уменьшается. В случае ошибки изменения сбрасываются.

Листинг 3.3 – Создание заказа

```
def create_order(self):
    try:
        client_id = self.client_combo.currentData()
        product_id = self.product_combo.currentData()
        quantity = self.quantity_input.text().strip()

        if not client_id or not product_id or not quantity:
            QMessageBox.warning(self, "Ошибка", "Заполните все
поля")
            return

        quantity = int(quantity)
        if quantity <= 0:
            QMessageBox.warning(self, "Ошибка", "Количество
должно быть больше 0")
            return
        product =
self.session.query(Product).filter_by(id=product_id).first()
        if product.stock_quantity < quantity:
            QMessageBox.warning(self, "Ошибка", f"Недостаточно
товара на складе. В наличии: {product.stock_quantity}")
            return

        new_order = ClientOrder(
            client_id=client_id,
            order_date=datetime.now().date(),
            status=OrderStatus.created
        )
        self.session.add(new_order)
        self.session.flush()
```

```

        order_item = OrderItem(
            order_id=new_order.id,
            product_id=product_id,
            quantity=quantity,
            price=product.price * quantity
        )
        self.session.add(order_item)

        product.stock_quantity -= quantity

        self.session.commit()
        QMessageBox.information(self, "Успех", "Заказ успешно
создан")
        self.accept()
    except ValueError:
        QMessageBox.warning(self, "Ошибка", "Количество должно
быть числом")
    except Exception as e:
        self.session.rollback()
        QMessageBox.critical(self, "Ошибка", f"Ошибка при
создании заказа: {e}")

```

При наличии нужных разрешений роли пользователь может создавать отчёт. Метод `generate_clients_report`, представленный на листинге 3.4 формирует отчёт, содержащий список всех клиентов предприятия. Отчёт включает информацию о клиентах, таких как их тип (физическое или юридическое лицо), контактные данные и дополнительные реквизиты для юридических лиц (ИНН, ОГРН, КПП).

Листинг 3.4 – Генерация отчёта по клиентам

```

def generate_clients_report(self):
    try:
        current_date = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"clients_report_{current_date}.pdf"
        doc = SimpleDocTemplate(filename, pagesize=A4)
        elements = []
        styles = getSampleStyleSheet()

        # Определяем стили с использованием шрифта DejaVuSans
        styles.add(ParagraphStyle(name='CustomTitle',
fontName='DejaVuSans-Bold', fontSize=14, leading=16, alignment=1))
        styles.add(ParagraphStyle(name='Normal',
fontName='DejaVuSans', fontSize=10, leading=12))

        elements.append(Paragraph("Отчёт по клиентам",
styles['CustomTitle']))
        elements.append(Paragraph(f"Дата формирования:
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}", styles['Normal']))
        elements.append(Paragraph("<br/><br/>",
styles['Normal']))

        data = [["ID", "Тип клиента", "Телефон", "Email", "ИНН",
"ОГРН", "КПП"]]

```

```

        clients =
self.session.query(Client).filter_by(is_deleted=False).all()
        for client in clients:
            row = [
                str(client.id),
                client.client_type.value,
                client.phone or "",
                client.email or "",
                client.legal_entity_client.inn if
client.client_type == ClientType.legal_entity and
client.legal_entity_client else "",
                client.legal_entity_client.ogrn if
client.client_type == ClientType.legal_entity and
client.legal_entity_client else "",
                client.legal_entity_client.kpp if
client.client_type == ClientType.legal_entity and
client.legal_entity_client else ""
            ]
            data.append(row)

```

```
table = Table(data)
```

Продолжение листинга 3.4

```

table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
    ('FONTNAME', (0, 0), (-1, 0), 'DejaVuSans-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 12),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('TEXTCOLOR', (0, 1), (-1, -1), colors.black),
    ('FONTNAME', (0, 1), (-1, -1), 'DejaVuSans'),
    ('FONTSIZE', (0, 1), (-1, -1), 10),
    ('GRID', (0, 0), (-1, -1), 1, colors.black),
]))
elements.append(table)

doc.build(elements)
QMessageBox.information(self, "Успех", f"Отчёт успешно
сформирован: {filename}")
except Exception as e:
    QMessageBox.critical(self, "Ошибка", f"Ошибка при
формировании отчёта: {e}")

```

Также есть возможность формирования отчёта по заказам, связанных, с выбранным клиентом или по всем заказам. Код метода `generate_orders_report` представлен на листинге 3.5.

Листинг 3.5 – Генерация отчёта по клиентам

```

# user_interface.py
def generate_orders_report(self):
    try:
        current_date = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"orders_report_{current_date}.pdf"

```



```

doc = SimpleDocTemplate(filename, pagesize=A4)
elements = []
styles = getSampleStyleSheet()

# Определяем стили с использованием шрифта DejaVuSans
styles.add(ParagraphStyle(name='CustomTitle',
fontName='DejaVuSans-Bold', fontSize=14, leading=16, alignment=1))
styles.add(ParagraphStyle(name='Normal',
fontName='DejaVuSans', fontSize=10, leading=12))

elements.append(Paragraph("Отчёт по заказам",
styles['CustomTitle']))
elements.append(Paragraph(f"Дата формирования:
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}", styles['Normal']))
elements.append(Paragraph("<br/><br/>",
styles['Normal']))

selected_row = self.client_table.currentRow()
if selected_row >= 0:
    client_id = int(self.client_table.item(selected_row,
0).text())

```

Продолжение листинга 3.5

```

    client =
self.session.query(Client).filter_by(id=client_id).first()
    elements.append(Paragraph(f"Клиент: {client.email}",
styles['Normal']))
    orders =
self.session.query(ClientOrder).filter_by(client_id=client_id).all()
    else:
        elements.append(Paragraph("Все заказы",
styles['Normal']))
        orders = self.session.query(ClientOrder).all()
        data = [["ID", "Дата заказа", "Статус", "Клиент",
"Товар", "Количество", "Цена"]]
        for order in orders:
            order_item =
self.session.query(OrderItem).filter_by(order_id=order.id).first()
            if order_item:
                product =
self.session.query(Product).filter_by(id=order_item.product_id).first(
)

                row = [
                    str(order.id),
                    str(order.order_date) if order.order_date
else "",
                    order.status.value,
                    str(order.client_id),
                    product.name if product else "",
                    str(order_item.quantity),
                    str(order_item.price)
                ]
                data.append(row)
table = Table(data)

```

```

table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
    ('FONTNAME', (0, 0), (-1, 0), 'DejaVuSans-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 12),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('TEXTCOLOR', (0, 1), (-1, -1), colors.black),
    ('FONTNAME', (0, 1), (-1, -1), 'DejaVuSans'),
    ('FONTSIZE', (0, 1), (-1, -1), 10),
    ('GRID', (0, 0), (-1, -1), 1, colors.black),
]))
elements.append(table)

doc.build(elements)
QMessageBox.information(self, "Успех", f"Отчёт успешно
сформирован: {filename}")
except Exception as e:
    QMessageBox.critical(self, "Ошибка", f"Ошибка при
формировании отчёта: {e}")

```